

Talk 4: QChem calculations with ORCA and QUICHE



Tutorial Roadmap

Room 717B

Tuesday September 15, 2026

10.00am

10.45am

12.00pm

1.00pm

1.45pm

2.30pm

1 Intro to
quantum
chemistry

2 From
chemistry to
quantum
computers

Lunch

3 Circuit
optimisations
and resource
estimates

4 Automating
resource
estimation on
software

Join in with Google Colab

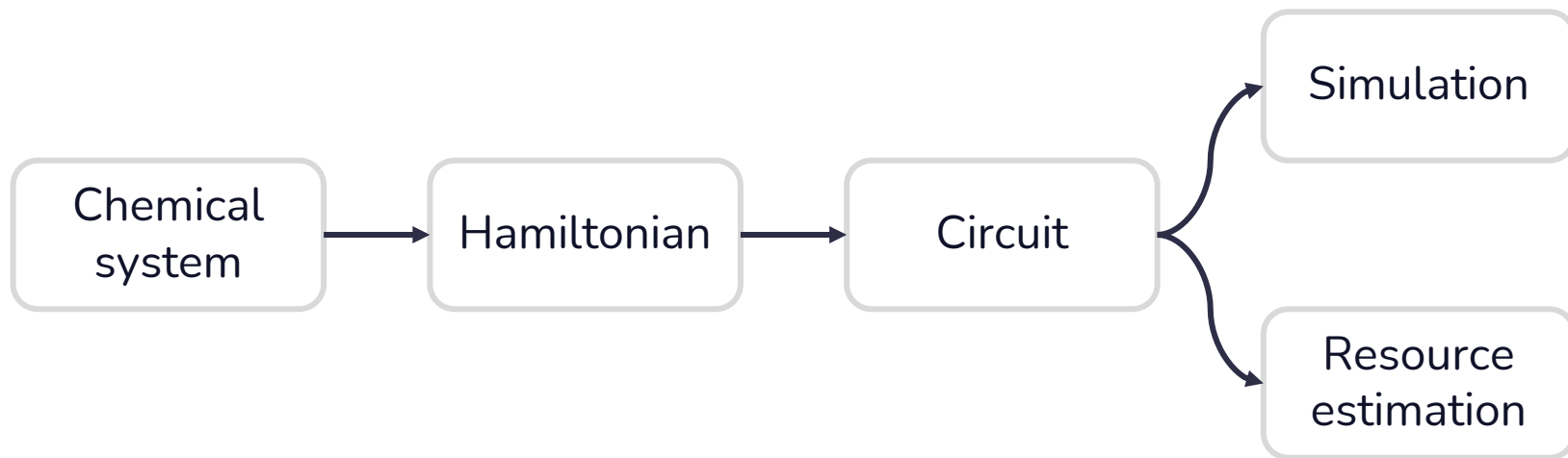


Link: <https://colab.research.google.com/drive/1jckxRkdL7fHAOfBj3sIH3vcCS3ldBtnX?usp=sharing>

Overview

- Set up an electronic structure calculation in ORCA
- Define the same calculation programmatically using the ORCA Python Interface
- Map an electronic Hamiltonian to a qubit Hamiltonian
- Generate quantum phase estimation circuits
- Simulate the phase estimation calculation
- Analyse resource costs across different algorithmic choices

The pipeline



Anatomy of an ORCA input and output

orca-calculation.inp

```
!RHF
!ST0-3G

* xyz 0 1
H  0.000 0.000 0.000
H  0.741 0.000 0.000
*

%casscf
    nel 2
    norb 2
end
```

Anatomy of an ORCA input and output

Simple keywords:

- Start with an exclamation mark (!)
- Set calculation type: Energy, MD, Opt, etc.
- Choose method: HF, DFT, MM, etc.
- Select an existing basis set: ST0-3G, def2-SVP, cc-pVTZ, etc.

orca-calculation.inp

```
!RHF
!ST0-3G

* xyz 0 1
H  0.000 0.000 0.000
H  0.741 0.000 0.000
*

%casscf
    nel 2
    norb 2
end
```

Anatomy of an ORCA input and output

Geometry block:

- Separated by stars (*)
- Set coordinate format, charge and multiplicity ($2S + 1$)
- Define each atom in the molecule
- Can also be loaded from a separate file, e.g. with .xyz file

orca-calculation.inp

```
!RHF
!STO-3G

* xyz 0 1
H  0.000 0.000 0.000
H  0.741 0.000 0.000
*

%casscf
    nel 2
    norb 2
end
```

Anatomy of an ORCA input and output

Input blocks:

- Separated by % and end
- Provide method specific configuration
- Wide variety, but common examples include: `basis`, `casscf`, `geom`, `output`, etc.

`orca-calculation.inp`

```
!RHF
!STO-3G

* xyz 0 1
H  0.000 0.000 0.000
H  0.741 0.000 0.000
*

%casscf
    nel 2
    norb 2
end
```

[illegible]

Anatomy of an ORCA input and output

Each simulation will generate a few different output files with specific purposes:

- `.out` file: calculation output file
- `.gbw` file: geometry, basis set and wavefunction data
- `.densities` file: density matrices
- `.property` file: computed physical and chemical properties

```
=====
                        WARNINGS
Please study these warnings very carefully!
=====
```

```
-----
SCF SETTINGS
-----
```

```
-----
ENERGY COMPONENTS
-----
```

```
*****
* Single Point Calculation *
*****
```

The OPI side of the story

OPI combines the power of ORCA with convenient Python scripting.

Closely resembles the input file format with added convenience and structure of the programmatic interface.

opi-calculation.py

```
structure = Structure(  
    atoms=[  
        Atom(element="H", coordinates=[0.000, 0.000, 0.000]),  
        Atom(element="H", coordinates=[0.741, 0.000, 0.000]),  
    ],  
    charge=0,  
    multiplicity=1,  
)  
  
calc = Calculator(basename="demo")  
calc.structure = structure  
calc.input.add_simple_keywords(ShellType.RHF, BasisSet.STO_3G)  
calc.input.add_blocks(BlockCasscf(nel=2, norb=2))  
  
calc.write_input()  
status = calc.run()
```

The OPI side of the story

Similarly the structured `Output` object provides structured access to results and data from each calculation.

Enables easy status checking, as well as efficient getters for a range of properties, such as energies, integrals, population analysis results, etc.

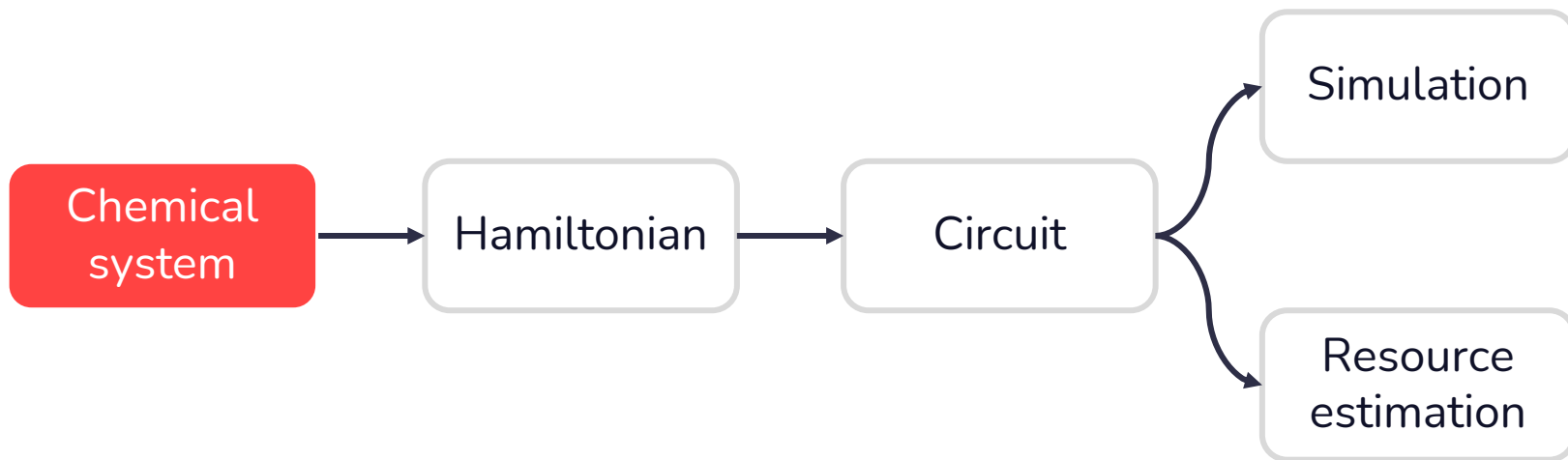
```
output = calc.get_output()  
output.parse()
```

```
output.get_final_energy()
```

```
output.get_int_overlap()
```

```
output.get_fcidump()
```

The pipeline



From chemistry to qubits

To construct a qubit Hamiltonian for QPE:

1. output the electronic integrals defining the second quantisation Hamiltonian from ORCA using an FCIDUMP
2. convert each fermionic operator into a qubit operator using a fermion-to-qubit mapping

hydrogen.fcidump

```
&FCI NORB= 2,NELEC= 2,MS2= 0,  
  ORBSYM=1,1,  
  ISYM= 1,  
&END  
  0.714139282456140    0 0 0 0  
 -1.252705257753715    1 1 0 0  
 -0.475697705661824    2 2 0 0  
  0.674565096014104    1 1 1 1  
  0.181266416982487    2 1 2 1  
  0.663537594129644    2 2 1 1  
  0.697467384335182    2 2 2 2
```

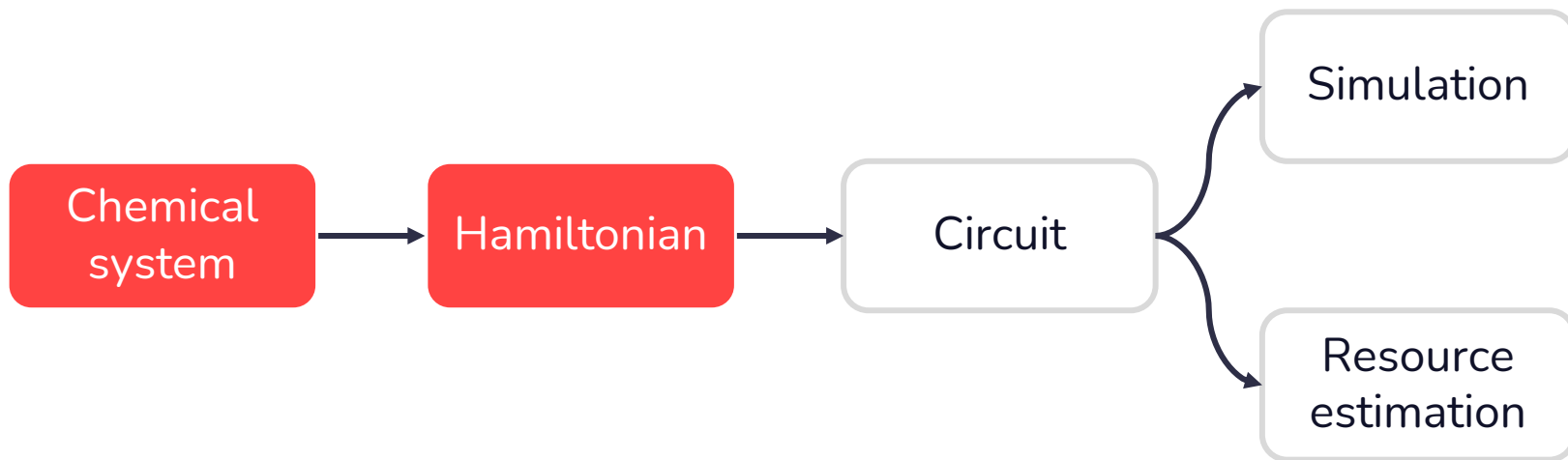
From chemistry to qubits

To construct a qubit Hamiltonian for QPE:

1. output the electronic integrals defining the second quantisation Hamiltonian from ORCA using an FCIDUMP
2. convert each fermionic operator into a qubit operator using a fermion-to-qubit mapping

```
integrals = fcidump.read("hydrogen.fcidump")  
hamiltonian = integrals.to_qubit_hamiltonian(  
    Mapping.JordanWigner  
)
```

The pipeline



Picking the subroutines

The QPE algorithm is made up of three key subroutine choices:

- Initial state preparation
- Hamiltonian simulation
- Phase estimation

```
state = HartreeFockState.closed_shell(  
    electrons=num_electrons,  
    spin_orbitals=num_spin_orbitals,  
)
```

```
simulation_algorithm = Simulation  
    ├── .Trotter  
    ├── .Qubitised  
    └── .QDRIFT
```

```
estimation_algorithm = Estimation  
    ├── .Textbook  
    ├── .Kitaev  
    ├── .Iterative  
    └── .Naive
```

Setting the error budget

Last step before getting our circuit is defining the error budget for each of its components.

- Estimation: target precision for the final energy
- Simulation: error for each instance of Hamiltonian simulation
- Rotations: error budget for synthesising arbitrary rotations into fault-tolerant gates
- State preparation: error preparing the initial state and its overlap with the exact target state

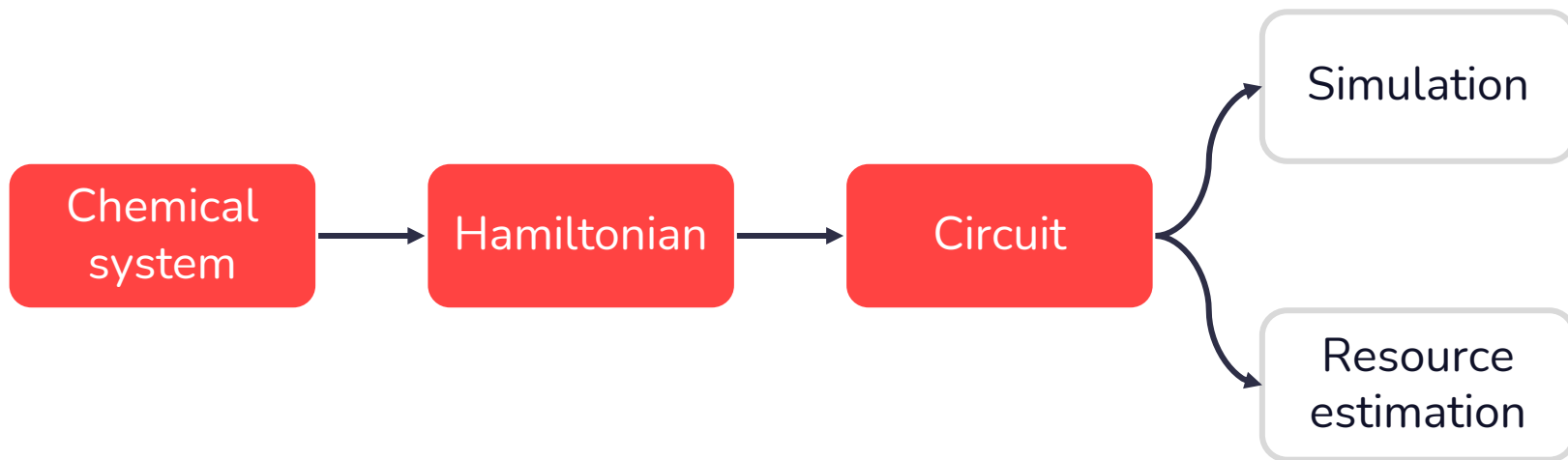
```
error_budget = Errors(  
    estimation=1e-3,  
    simulation=1e-4,  
    rotations=1e-5,  
    state_prep=1e-3,  
    overlap=0.90,  
)
```

Generating the circuit

Bringing it all together we can construct a QPESpec object: a complete specification of our target phase estimation calculation including key routines and error budgets.

```
qpe_calculation = QPESpec(  
    hamiltonian=electronic_hamiltonian,  
    state_prep=state,  
    error_budget=error_budget,  
    simulation=Simulation.Qubitised,  
    algorithm=PhaseEstimation.Textbook,  
)
```

The pipeline



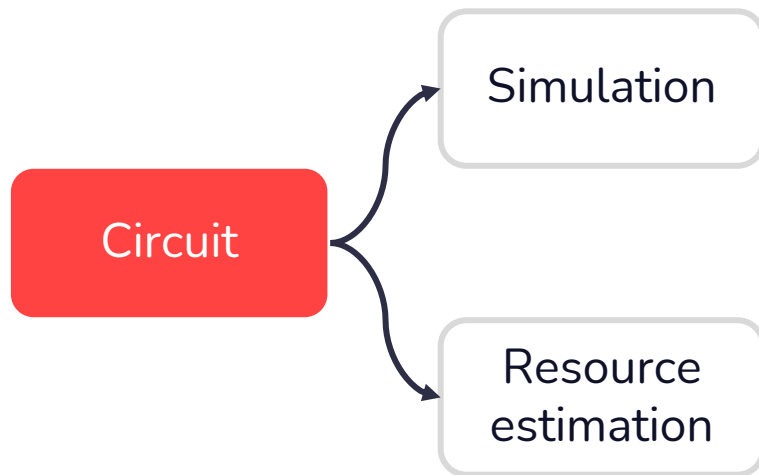
Choosing a backend

Simulation:

- What do the results from this calculation look like?
- Run on statevector simulator, small systems only.

Resource estimation:

- What would this cost to run?
- Generate hierarchical representation in terms of subroutines



Using each backend

Simulation:

Generate the statevector simulation and evaluate using the compiled simulator bindings.

```
with QuESTEnv():  
    qureg = Qureg(qpe_spec.num_qubits)  
    qpe_spec.to_quest().evaluate(qureg)
```

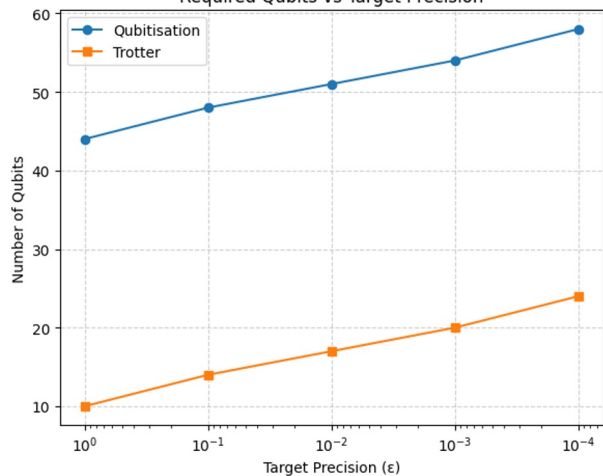
Resource estimation:

Generate the algorithm's complete call graph and analyse it's qubit and gate properties.

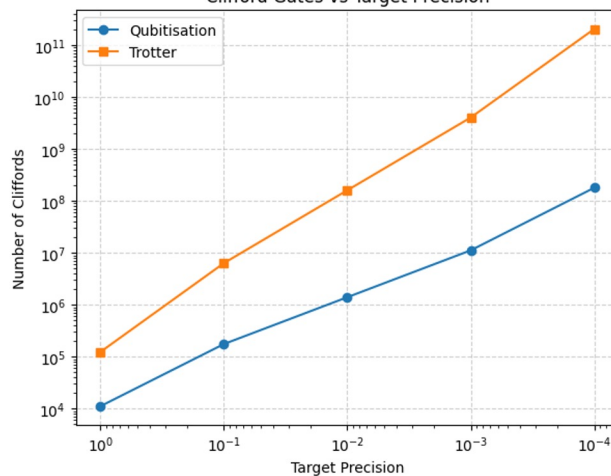
```
bloq = qpe_spec.get_composite_bloq()  
qubits = logical_qubit_resources(bloq)  
gates = logical_gate_resources(bloq)
```

Some example results

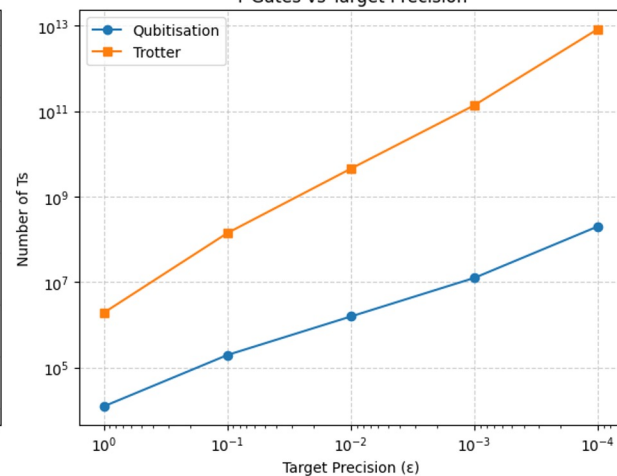
Required Qubits vs Target Precision



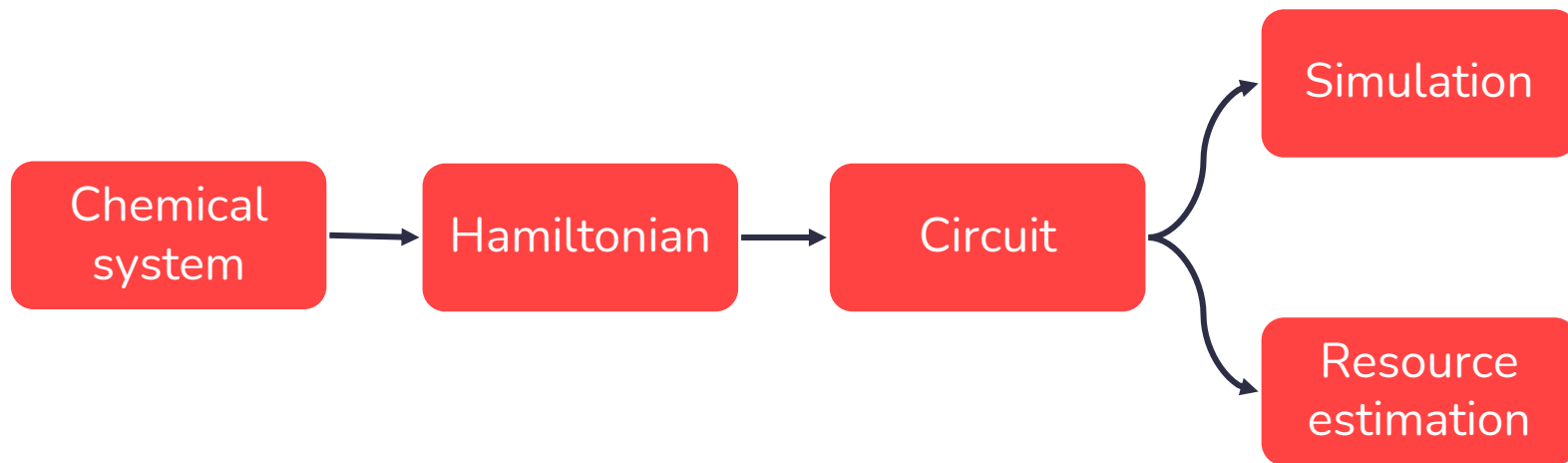
Clifford Gates vs Target Precision



T Gates vs Target Precision



The pipeline



The QUICHE package

Where to get QUICHE:

- On Github: github.com/Quantum-Motion/quiche
- On PyPI: `pip install pyquiche`

Coming soon:

- Electronic hamiltonian factorisations (DFTHC)
- Improved error budgeting
- Sampling/ shot cost accounting
- ... and much more

Summary

- Defined the electronic structure problem and the quantum chemistry that underpins it
- Learned how quantum computing can be used as part of the solution
- Explored different quantum algorithm implementations, their pros and cons
- Saw how it all comes together from ORCA to resource estimates and simulations with QUICHE